SecurePlan Mobile — Calendario

Documentación técnica de los Sprints 1 y 2

Explicación, sprint por sprint, de cómo se cumplió lo pedido por la Guía Oficial del Proyecto Integrador: patrón Singleton, seguridad de las credenciales, base de datos y funcionalidad de la agenda.

Proyecto	SecurePlan Mobile (Calendario)
Asignatura	Desarrollo Móvil Integral
Stack	React Native + Expo SDK 58 / Node.js + Express 5 / PostgreSQL 16
Sprints cubiertos	Sprint 1 (base y autenticación) y Sprint 2 (agenda)
Fecha	2026-10-15

Índice de contenido

1. Objetivo del documento
2. Arquitectura general
3. Sprint 1 — Base móvil, arquitectura y autenticación segura
4. Sprint 2 — Core de la agenda (módulo de eventos)
5. La base de datos: qué se creó en cada sprint
6. Cómo ejecutar el proyecto
7. Conclusión

1. Objetivo del documento

Este documento explica de forma sencilla qué pidió la Guía Oficial del Proyecto Integrador en los Sprints 1 y 2, y cómo se resolvió cada punto en el código. Para cada requisito se indica qué pedía la guía, qué se hizo en el proyecto y en qué archivo está.

Cómo leer las referencias: las rutas se escriben relativas a la raíz del repositorio, por ejemplo `backend/src/config/database.js`.

2. Arquitectura general

La aplicación está separada en dos partes que se comunican por HTTP/JSON: una **app móvil** (React Native + Expo) y un **backend API REST** (Node.js + Express) que guarda la información en **PostgreSQL**. El frontend no conoce la base de datos y el backend no conoce la interfaz: el contrato entre ambos es la API.

```
Calendario/
■■■ backend/                  API REST (Node + Express + PostgreSQL)
■   ■■■ src/config/           entorno + conexión a BD (Singleton)
■   ■■■ src/controllers/      lógica de cada endpoint
■   ■■■ src/middleware/       autenticación JWT + manejo de errores
■   ■■■ src/models/           acceso a datos + hashing
■   ■■■ src/routes/           definición de rutas
■   ■■■ src/services/         firma de tokens (JWT)
■   ■■■ src/validators/       esquemas de validación (Zod)
■   ■■■ sql/init.sql          esquema de base de datos
■   ■■■ docker-compose.yml    PostgreSQL en desarrollo
■■■ frontend/                 App móvil (React Native + Expo)
    ■■■ src/                  domain / data / navigation / presentation
                              services / theme
```

3. Sprint 1 — Base móvil, arquitectura y autenticación segura

La meta del Sprint 1 fue dejar listos los cimientos: la conexión a la base de datos con el patrón Singleton, el registro/login con contraseñas cifradas y tokens JWT, la arquitectura de carpetas del frontend, la navegación protegida y los formularios de Login/Registro.

Lo que pedía la guía	Cómo se cumplió en el proyecto	Dónde
Conexión a BD con patrón Singleton	Una única instancia de conexión a PostgreSQL.	config/database.js
Módulo de usuarios con cifrado a contraseñas	Contraseñas cifradas con bcrypt (12 rondas).	models/user.model.js
Generación de tokens JWT firmados	Firma y verificación del token con expiración.	services/token.service.js
Arquitectura limpia de directorios	Carpetas separadas por responsabilidad.	frontend/src/*
Navegación nativa protegida (pilas separadas)	Pila según autenticado vs. no autenticado.	navigation/RootNavigator.js
Formularios de Login/Registro	Pantallas conectadas al backend.	presentation/screens/*

3.1. Patrón Singleton: cómo se usó para la base de datos

La guía pide configurar la conexión al servidor de base de datos con el **patrón Singleton**. La idea del patrón es que exista **una sola** conexión (un solo Pool) compartida por toda la aplicación.

En este proyecto el Singleton vive en `backend/src/config/database.js`. El archivo define la clase `DatabaseConnection`. El constructor guarda la primera instancia en la propiedad estática `instance` y, a partir de ahí, siempre devuelve esa misma instancia. El archivo termina con `module.exports = new DatabaseConnection()`, de modo que todos los módulos que lo importan reciben el mismo objeto y, por tanto, el mismo Pool.

```
# backend/src/config/database.js (núcleo del Singleton)
class DatabaseConnection {
  constructor() {
    if (!DatabaseConnection.instance) {   // ¿ya existe?
      DatabaseConnection.instance = this;
      this.pool = null;
    }
    return DatabaseConnection.instance;    // siempre la misma instancia
  }
  async connect() { /* crea el Pool una sola vez */ }
  query(text, params) { return this.getPool().query(text, params); }
}
module.exports = new DatabaseConnection();  // instancia única exportada
```

El Pool se crea una sola vez dentro de `connect()` (máximo 10 conexiones). Al arrancar, el servidor llama a `db.connect()` antes de aceptar tráfico: si la base de datos no responde, el proceso termina con error y el problema se ve de inmediato. Los modelos (`user.model.js` y `event.model.js`) nunca crean conexiones propias: siempre usan `db.query(...)`.

3.2. Módulo de usuarios con cifrado asíncrono

La guía prohíbe guardar contraseñas en texto plano. Por eso el modelo de usuario cifra la contraseña con **bcrypt** y 12 rondas de sal antes de guardarla. El correo se normaliza a minúsculas y el método `toJSON()` excluye el hash, así que la contraseña cifrada nunca puede salir en una respuesta de la API.

```
# backend/src/models/user.model.js (fragmento)
const SALT_ROUNDS = 12;
static async hashPassword(plainPassword) {
  const salt = await bcrypt.genSalt(SALT_ROUNDS);
  return bcrypt.hash(plainPassword, salt);
}
verifyPassword(plainPassword) {
  return bcrypt.compare(plainPassword, this.password);
}
```

3.3. Tokens JWT firmados

El servicio de tokens firma un JWT con el identificador y el correo del usuario, y con expiración (por defecto 1 hora). El middleware de autenticación exige la cabecera `Bearer`, verifica el token y deja el usuario decodificado en `req.user`. Ese dato es la fuente de verdad del propietario en todas las operaciones protegidas.

```
# backend/src/services/token.service.js
function sign(user) {
  return jwt.sign({ sub: user.id, email: user.email },
                  env.jwt.secret, { expiresIn: env.jwt.expiresIn });
}

# backend/src/middleware/auth.middleware.js (fragmento)
const [scheme, token] = header.split(' ');
if (scheme !== 'Bearer' || !token) return res.status(401).json({...});
req.user = tokenService.verify(token);
```

3.4. Frontend: arquitectura limpia y navegación protegida

El frontend (`frontend/src`) está organizado por responsabilidad, de forma que la interfaz no mezcla acceso a datos ni utilidades.

Capa	Responsabilidad	Ejemplos
domain/	Entidades y utilidades puras (sin React)	entities/Event.js, utils/date.js
data/	Cliente HTTP y repositorios	api/client.js, repositories/*
services/	Servicios del dispositivo	secureStorage.js
navigation/	Pilas de navegación	RootNavigator, AuthStack, AppStack
presentation/	Pantallas, componentes y estado	screens/*, components/*, context/*
theme/	Tokens de diseño	theme.js

La **navegación protegida** se resuelve en `RootNavigator.js`, que decide qué mostrar según la sesión: mientras se restaura el token muestra la pantalla de carga; si hay sesión muestra `AppStack`, y si no, `AuthStack`. Un usuario no autenticado **no puede** llegar a las pantallas protegidas porque ni siquiera están montadas.

```
# frontend/src/navigation/RootNavigator.js (fragmento)
if (isRestoring) return <SplashScreen />;
{isAuthenticated
  ? <EventsProvider><AppStack /></EventsProvider>
  : <AuthStack />}
```

3.5. Criterio de seguridad del Sprint 1

- **Contraseñas:** cifradas con bcrypt (12 rondas); nunca en texto plano.
- **Token JWT:** guardado en **Expo SecureStore** (`services/secureStorage.js`), que cifra el valor con el Keychain (iOS) / Keystore (Android). **No** se usa AsyncStorage.

4. Sprint 2 — Core de la agenda (módulo de eventos)

La meta del Sprint 2 fue agregar el corazón de la aplicación: el CRUD completo de eventos (crear, leer, editar y borrar) protegido por autenticación, la validación estricta de los datos y la vista de calendario en el móvil.

Lo que pedía la guía	Cómo se cumplió en el proyecto	Dónde
Endpoints CRUD protegidos por middleware	Rutas de eventos detrás del middleware de autenticación.	routes/events.routes.js
Validación estricta con esquemas (Zod)	Reglas de fecha, hora, categoría y prioridad.	validators/event.schema.js
Vista principal con calendario	Pantalla de agenda con calendario y lista del día.	screens/AgendaScreen.js
Interceptores HTTP globales para el token	El cliente agrega el JWT a la cabecera Bearer automáticamente.	data/api/client.js
Validación del lado del servidor con propiedad	Cada usuario solo ve y modifica sus eventos; el dueño sale del token, no del cliente.	controllers + event.model.js

4.1. Backend: CRUD protegido por middleware

Todas las rutas de eventos pasan primero por el middleware de autenticación. Así, cualquier petición sin un JWT válido se rechaza antes de llegar al controlador.

```
# backend/src/routes/events.routes.js
router.use(authenticate);          // protege TODAS las rutas de eventos
router.get('/',    eventsController.list);
router.get('/:id', eventsController.getOne);
router.post('/',   eventsController.create);
router.put('/:id', eventsController.update);
router.delete('/:id', eventsController.remove);
```

4.2. Backend: validación estricta de datos con Zod

La guía pide validar los datos contra inyecciones usando esquemas. Los esquemas están en `validators/event.schema.js` y usan `.strict()`, que rechaza cualquier campo no permitido. Además validan el formato de fecha (AAAA-MM-DD) y hora (HH:MM) con expresiones regulares, restringen categoría y prioridad a una lista válida, y comprueban que la hora de fin sea posterior a la de inicio.

```
# backend/src/validators/event.schema.js (fragmento)
const createEventSchema = withTimeOrder(z.object(baseEvent).strict());
// withTimeOrder -> refine(data => data.startTime < data.endTime)
```

4.3. Frontend: vista principal con calendario

La pantalla principal (`AgendaScreen`) muestra un calendario mensual, la lista de eventos del día seleccionado y un botón para crear eventos. El formulario de crear/editar vive en `EventFormModal`. El estado de los eventos se centraliza en `EventsContext`, que agrupa los eventos por día y expone crear, editar y eliminar.

4.4. Frontend: interceptores HTTP globales

La guía pide interceptores globales que adjunten el JWT automáticamente. En `data/api/client.js` un interceptor de petición lee el token de SecureStore y agrega la cabecera `Authorization: Bearer ...`; un interceptor de respuesta borra el token si el servidor responde 401, forzando el cierre de sesión.

```
# frontend/src/data/api/client.js (fragmento)
apiClient.interceptors.request.use(async (config) => {
  const token = await getAuthToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

4.5. Criterio de seguridad del Sprint 2: mitigación de IDOR

IDOR ocurre cuando un usuario puede leer o modificar los datos de otro cambiando un identificador en la petición. Aquí se evita de dos maneras: el controlador toma el propietario de `req.user.sub` (que viene del JWT, nunca del cliente) y el modelo agrega `AND user_id = $n` a todas sus consultas. Si alguien intenta acceder al evento de otro, la consulta no lo encuentra y el servidor responde 404, sin revelar que el recurso existe.

```
# backend/src/models/event.model.js (fragmento anti-IDOR)
static async findByIdForUser(id, userId) {
  const { rows } = await db.query(
    `SELECT ${COLUMNS} FROM events WHERE id = $1 AND user_id = $2`,
    [id, userId]               // userId sale del token, no del cliente
  );
```

```
  return rows[0] ? new Event(rows[0]) : null;
}
```

5. La base de datos: qué se creó en cada sprint

Se eligió **PostgreSQL 16** por su tipado fuerte y soporte de transacciones. En desarrollo se levanta con **Docker Compose**: el archivo `sql/init.sql` se monta en el directorio que PostgreSQL ejecuta la primera vez que se inicializa, así que el esquema se crea solo al levantar el contenedor.

Sprint 1 — tabla `users`: guarda las cuentas. El campo `password` almacena el hash de bcrypt (nunca texto plano). El índice sobre `LOWER(email)` hace que el login busque sin distinguir mayúsculas y de forma rápida.

Sprint 2 — tabla `events`: guarda la agenda. Cada evento pertenece a un usuario mediante `user_id` (clave foránea con `ON DELETE CASCADE`, es decir, si se borra el usuario se borran sus eventos). El índice `(user_id, event_date)` acelera el listado por usuario y día.

```
-- backend/sql/init.sql
CREATE TABLE IF NOT EXISTS users (                    -- Sprint 1
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,                     -- hash bcrypt
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_users_email ON users (LOWER(email));

CREATE TABLE IF NOT EXISTS events (                   -- Sprint 2
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  event_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  category VARCHAR(20) NOT NULL DEFAULT 'other',
  priority VARCHAR(20) NOT NULL DEFAULT 'medium',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_events_user_date ON events (user_id, event_date);
```

Seguridad de las consultas: todas las consultas de la aplicación usan parámetros (`$1, $2, ...`) y nunca concatenan valores en el texto de la consulta, lo que evita la inyección SQL.

6. Cómo ejecutar el proyecto

6.1. Backend y base de datos

```
cd backend
cp .env.example .env
docker compose up -d      # levanta PostgreSQL (contenedor calendario-db)
npm install
npm run dev               # servidor en http://localhost:4000

curl http://localhost:4000/health   # verificación rápida
```

6.2. Frontend móvil

```
cd frontend
cp .env.example .env      # configurar EXPO_PUBLIC_API_URL
npm install
npx expo start            # servidor de desarrollo (Metro)
npx expo lint             # lint
npx tsc --noEmit          # typecheck
```

7. Conclusión

Los Sprints 1 y 2 quedaron implementados de acuerdo con la guía: el patrón Singleton está en `config/database.js` y es la única puerta de acceso al Pool de PostgreSQL; la base de datos crea la tabla de usuarios en el Sprint 1 y la de eventos en el Sprint 2, ambas con sus índices; y la seguridad se apoya en bcrypt, JWT guardado en SecureStore, consultas parametrizadas y la mitigación de IDOR filtrando siempre por `user_id` tomado del token.

Nota de versión: este documento se reorganizó para cubrir solo lo solicitado por la guía, sprint por sprint. Quedan pendientes los Sprints 3 (Strategy y Observer) y 4 (pruebas y auditoría).